

CSE233 Week 3: If Statements and Boolean Logic

The first conditional statement we will look at is the "if statement". An if statement executes a block of code *if* a given condition is true. If statements can optionally have an else block that only executes if the given condition is false. The condition for an if statement is a Boolean expression which evaluates to true or false. Similar to if statements are switch statements, which we'll also discuss this week.

If Statements

An if statement is a conditional statement that is used to execute a block of code if its given condition is true. Remember that in C++, a block of code is denoted by curly braces. A basic C++ if statement looks like the following:

```
if (condition) {  
    statement;  
}
```

If there is only a single statement for the if, the curly braces are optional:

```
if (condition)  
    statement;
```

If statements are started with the `if` keyword. It is followed by a set of parenthesis with the condition in them. The condition can be a value, a variable, or an expression. Either the single statement or a block follows the condition. I'd recommend always putting a block, even if you only have a single statement. It makes it easier to add additional statements to the if later, if you need to. Here is an example:

```
// declare and integer variable, x, whose value is 2
int x = 2;
// remember that % is the remainder. if the remainder
// of x divided by 2 is 0, x is even
if (x % 2 == 0) {
    std::cout << "x is even\n";
}
```

This handles printing a message if the value of x is even. But what if we want to print a message if x is odd? Then, we would need an else clause. To add an else to an if statement, place the `else` keyword after the block of the if. Then, put a block after the else to be executed. Else clauses do not have conditions. Example:

```
int x = 2;

if (x % 2 == 0) {
    std::cout << "x is even\n";
}
// the else comes immediately after the if's block
else {
    std::cout << "x is odd\n";
}
```

If there are multiple conditions that need to be checked, we can either nest if statements, or use the `else if` syntax to chain if statements together. Consider this example that determines if a number is positive, negative, or 0:

```
// first, we'll nest the if statements
if (x > 0) {
    std::cout << "x is positive\n";
}
else {
    // the second if statement goes inside the else block
    if (x < 0) {
        std::cout << "x is negative\n";
    }
    // we need another else
    else {
        std::cout << "x is 0\n";
    }
}

// now, we'll do the same thing with "else if"
```

```
if (x > 0) {
    std::cout << "x is positive\n";
}
// we put "if (condition)" immediately after "else"
else if (x < 0) {
    std::cout << "x is negative\n";
}
// then the final "else" block
else {
    std::cout << "x is 0\n";
}
```

The "else if" syntax is typically preferred since it reduces the level of nesting.

Variable Scopes

Variables in C++ are scoped to the block that they are in. This means that a variable declared in the block of an if statement only exists in that block.

Switch Statements

Switch statements can replace an else if chain with a more concise form. A switch statement can be used when:

1. The condition being tested are single integer-like values
2. There are a finite number of conditions, with a possible default action

If those two conditions are met, a switch statement is usable. If the condition is not an integer value, or if you need to test ranges, then switches cannot be used. A switch statement consists of the keyword `switch`, followed by a test expression in parentheses. This is followed by a block. The block of a switch is special, since it contains cases. Here is the switch syntax:

```
switch (test_expression) {
    case 1:
        statement;
        break;
    case 2:
        statement1;
        statement2;
        break;
```

```
    default:
        statement;
}
```

There are a few things to note. First, each case can have multiple statements, but these statements do not have to be wrapped in curly braces. Each case ends with a `break;` statement. This tells the compiler that you want to exit the switch statement. If you do not have a break statement, you will "fall through" to the next case. While there are times when this behavior is desirable, typically you want to break out of each case. In this class, you will always want to break out of each case. Finally, there is a default case. This is the last case in the switch, and is used to handle any input to the switch that does not match any of the cases. Since it is the last case, it does not need a break statement.

Below is an example "elevator" simulation using a switch:

```
int current_floor = 0;
int selection;
std::cout << "Enter floor (1-4): ";
std::cin >> selection;

// the test case for the switch is the user's floor selection
switch (selection) {
    // there is a case for each valid floor
    case 1:
        std::cout << "Moving to floor 1\n";
        current_floor = 1;
        break;
    case 2:
        std::cout << "Moving to floor 2\n";
        current_floor = 2;
        break;
    case 3:
        std::cout << "Moving to floor 3\n";
        current_floor = 3;
        break;
    case 4:
        std::cout << "Moving to floor 4\n";
        current_floor = 4;
        break;
    // the default case prints a message stating that the
    // selection is invalid
    default:
        std::cout << "Invalid selection\n";
}
```

Any switch can also be written as a chain of if-else if-else statements:

```
int current_floor = 0;
int selection;
std::cout << "Enter floor (1-4): ";
std::cin >> selection;

if (selection == 1) {
    std::cout << "Moving to floor 1\n";
    current_floor = 1;
}
else if (selection == 2) {
    std::cout << "Moving to floor 2\n";
    current_floor = 2;
}
else if (selection == 3) {
    std::cout << "Moving to floor 3\n";
    current_floor = 3;
}
else if (selection == 4) {
    std::cout << "Moving to floor 4\n";
    current_floor = 4;
}
else {
    std::cout << "Invalid selection\n";
}
```

Stacked Cases

If the same code needs to run for multiple cases, the case labels are stacked. The syntax `case 1, 2, 3:` is illegal. You need a separate case tag for each case. The example below prints a message based on the letter grade entered by the user:

```
char grade;
std::cout << "Enter your letter grade: ";
std::cin >> grade;

// internally, the char data type is an integer,
// so char can be used for switch cases as well
switch (grade) {
    // the "You passed" message will be printed
    // if the grade is A, B, C, or D
    case 'A':
    case 'B':
    case 'C':
    case 'D':
        std::cout << "You passed\n";
}
```

```

        break;
    case 'F':
        std::cout << "You failed\n";
        break;
    default:
        std::cout << "Invalid grade\n";
}

```

Boolean Logic

Boolean logic, also called Boolean algebra, are expressions that evaluate to either true or false. It is named for the mathematician George Boole. There are three main operations in Boolean logic: AND, OR, and NOT. The AND operation is a binary operation and is true iff (if and only if) both operands are true. The OR operation is a binary operation and is false iff both operands are false. The NOT operation is a unary operation, and simply negates its operand. I.E., NOT true is false and NOT false is true.

Truth Tables

Truth tables are used to visualize the possible outcomes of Boolean operations. Below is a truth table for two values, a and b, and the possible combinations of them:

								NOT	
				a	a	NOT	NOT	a	NOT
				AND	OR	(a	(a	AND	a OR
a	b	NOT	NOT	b	b	AND	OR	NOT	NOT
		a	b			b)	b)	b	b
true	true	false	false	true	true	false	false	true	true
true	false	false	true	false	true	true	false	false	true
false	true	true	false	false	true	true	false	false	true
false	false	true	true	false	false	true	true	true	true

There should be a column for each input variable and operation between the input variables. The number of rows scales exponentially with the number of input variables. In this truth table, there are two

input variables, a and b. a and b can have two possible values, true or false, and there are two input variables, so the number of rows is $2^2 = 4$. If we added a third input variable, we'd have $2^3 = 8$ rows. For four input variables, we'd have $2^4 = 16$ rows, and so on. Therefore, truth tables scale poorly when there are many input variables, but if you can minimize the number of input variables truth tables can be useful for determining the results of various Boolean expressions.

Precedence

Comparison operators have lower precedence than arithmetic operators. Arithmetic operators (+, -, *, /, etc.) follow the PEMDAS you should have learned in grade school. Next, the comparison operators are applied. The <, >, <=, >= are applied before == and !=. Next, AND is applied. Finally, OR is applied. This is true in most programming languages.

De Morgan's Laws

De Morgan's laws are a pair of equivalences used for simplifying boolean expressions. We'll skip the formal proof, and instead look to the truth table to demonstrate the validity of the laws.

1. NOT (a AND b) = (NOT a) OR (NOT b)
2. NOT (a OR b) = (NOT a) AND (NOT b)

If you look at the table, you'll see that for each row, the column for "NOT (a AND b)" has the same value as "NOT a OR NOT b". The same applies for "NOT (A OF B)" and "NOT a AND NOT b".

In modern programming, these are less relevant, as the compiler/interpreter will apply De Morgan's laws to make Boolean logic more efficient on its own. Nonetheless, remembering the laws can be useful when trying to think through what a Boolean expression is doing.

Numbers as Boolean Values

Many modern programming languages are based on C. C did not have a Boolean type originally. Instead, it used 0 for false and 1 for true. In reality, any numeric value that is not 0 is interpreted as true. The same is true in C++. For example:

```
if (5) {  
    std::cout << "5 is true!\n";  
}
```

The same is true for floating point numbers:

```
if (-4.2) {  
    std::cout << "-4.2 is true!\n";  
}
```

Examples

Here are some examples using if statements and switch statements.

Login

```
#include <iostream>  
#include <string>  
  
// obviously, this isn't accurate to a real login system  
// it's just to demonstrate if statements  
int main() {  
    // string variables to hold username and password  
    std::string username;  
    std::string password;  
  
    // read username and password  
    std::cout << "Enter username: ";  
    std::cin >> username;  
    std::cout << "Enter password: ";  
    std::cin >> password;  
  
    // if the username is admin, print admin login message  
    if (username == "admin" && password == "password") {  
        std::cout << "Admin login successful\n";  
    }  
    // if the username is admin and the password was not "password"  
    // the password is incorrect  
    else if (username == "admin") {  
        std::cout << "Admin login unsuccessful. Incorrect password.\n";  
    }  
    // if user is the username and "password" is the password  
    // we'll consider this a valid login  
    else if (username == "user" && password == "password") {  
        std::cout << "User login successful\n";  
    }  
    // if the username is "user" and the password is not "password"
```



```

// the password is not correct
else if (username == "user") {
    std::cout << "Admin login unsuccessful. Incorrect password.\n"
}
// any username that is not "admin" or "user" is invalid
else {
    std::cout << "Unknown user: " << username << "\n";
}
}

```

Vending Machine

```

#include <iostream>

int main() {
    // print out the options and prices
    std::cout << "A1: Plain Chips $1.50 ----- ";
    std::cout << "A2: BBQ Chips $1.50 ----- ";
    std::cout << "A3: Jalepeno Chips $1.50 --\n";
    std::cout << "B1: PB Cups $1.75 ----- ";
    std::cout << "B2: Chocolate Bar $2.50 --- ";
    std::cout << "B3: Cookie $2.00 ----- \n";
    std::cout << "C1: Trail Mix $2.25 ----- ";
    std::cout << "C2: Protein Bar $3.50 ----- ";
    std::cout << "C3: Granola Bar $2.50 ----- \n";

    // get money from user
    double money;
    std::cout << "Enter money: $";
    std::cin >> money;

    char row;
    int column;

    // read the row and column of the selection
    std::cout << "Enter row: ";
    std::cin >> row;
    std::cout << "Enter column: ";
    std::cin >> column;

    // used to store the item price to compare later
    double item_price = 0;

    // use switch to find item price
    switch (row) {
        // first, the row is checked
        // the character 'A' is different than 'a'
        // so both can be checked if we want to allow lowercase
        case 'A':
        case 'a':
    }
}

```

```

// each row case has a nested switch that checks the column
switch (column) {
    case 1:
        // each case sets the price
        item_price = 1.5;
        break;
    case 2:
        item_price = 1.5;
        break;
    case 3:
        item_price = 1.5;
        break;
    default:
        // default case handles invalid input
        std::cout << "Invalid column\n";
}
break;
case 'B':
case 'b':
    switch (column) {
        case 1:
            item_price = 1.75;
            break;
        case 2:
            item_price = 2.5;
            break;
        case 3:
            item_price = 2.0;
            break;
        default:
            std::cout << "Invalid column\n";
    }
    break;
case 'C':
case 'c':
    switch (column) {
        case 1:
            item_price = 2.25;
            break;
        case 2:
            item_price = 3.5;
            break;
        case 3:
            item_price = 2.5;
            break;
        default:
            std::cout << "Invalid column\n";
    }
    break;
default:
    std::cout << "Invalid row\n";

```

```

}

// if the item_price is still 0,
// the selection was invalid
if (item_price == 0) {
    std::cout << "Invalid selection, try again later\n";
}
// check if the amount of money entered is more
// than the price of the item selected
else if (item_price < money) {
    double change = money - item_price;
    std::cout << "Your change is $" << change << std::endl;
}
// otherwise, they can't afford it
else {
    double shortage = item_price - money;
    std::cout << "You are $" << shortage << " short\n";
}

return 0;
}

```

Conclusion

This week we discussed if statements. If statements are conditional statements that execute a block of code only if their condition is true. If statements can have an optional else clause, which executes a block of code if the condition is false. There are also "else if" clauses, that allow multiple conditions to be used within the same if statement. We also discussed Boolean logic for constructing complex conditions. Chains of else if clauses can be replaced with switch statements when the values being tested are integer-like (`int`, `char`, `long`, etc.) values as a condition.